Practical Work 2: RPC File Transfer

Student Name

December 5, 2025

1 Objective

The objective of this practical work is to upgrade the TCP file transfer system into a Remote Procedure Call (RPC) based file transfer system. The client uses a high-level remote method instead of directly handling socket communication.

2 RPC Service Design

The RPC service provides a remote function named:

`upload_file(filename, data)`

The client calls this function to transmit a file to the server. The server executes the function, stores the received file locally, and returns a success message to the client.

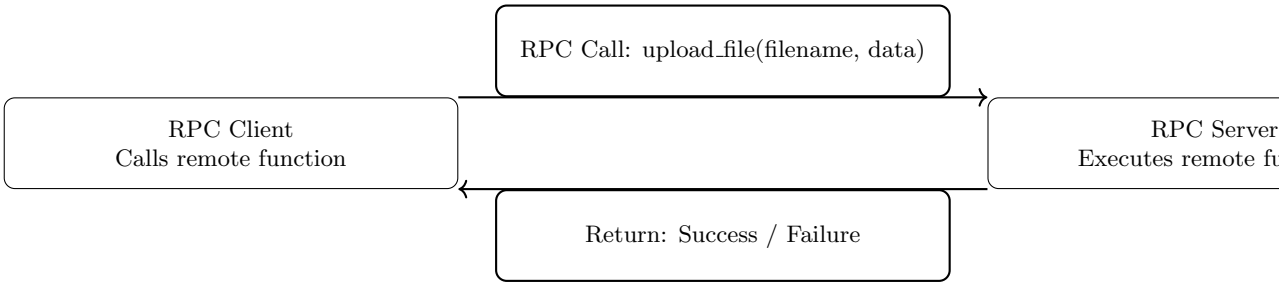

Figure 1: RPC Service Design for File Transfer

3 System Organization

The system is composed of two main components: the RPC client and the RPC server. The client reads the file and sends it to the server using an XML-RPC call over TCP. The server receives the transmitted data and writes it to disk.

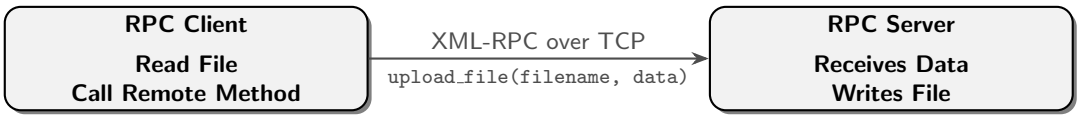

Figure 2: RPC Service Design for File Transfer

4 Implementation

4.1 Server Code (server.py)

```python
from xmlrpc.server import SimpleXMLRPCServer
import base64

def upload_file(filename, data):
    file_data = base64.b64decode(data)
    with open(filename, "wb") as f:
        f.write(file_data)
    return "File received successfully"

server = SimpleXMLRPCServer(("0.0.0.0", 8000))
server.register_function(upload_file, "upload_file")
print("RPC Server running on port 8000...")
server.serve_forever()
```

4.2 Client Code (client.py)

```python
import xmlrpc.client
import base64

proxy = xmlrpc.client.ServerProxy("http://localhost:8000/")
filename = "test.txt"

with open(filename, "rb") as f:
    data = f.read()

encoded_data = base64.b64encode(data).decode()
result = proxy.upload_file(filename, encoded_data)
print(result)
```

5 Role Distribution

The client is responsible for reading the file and invoking the remote function. The server executes the remote function and saves the received file. The RPC middleware handles data serialization, transmission, and decoding.

6 Conclusion

This project demonstrates how RPC simplifies distributed file transfer by replacing manual TCP socket programming with a high-level remote procedure call mechanism. The implementation improves modularity, readability, and ease of development.